

✓ AI-Based Network Pathfinding and Attack Simulation

COMSATS University Islamabad — CSC 262 Artificial Intelligence

Student: Ahsan Khalid **Reg. No.:** FA24-BCS-115 **Section:** BCS-4C

Instructor: Ms. Zeenat Zulfiqar **Date:** May 28, 2026

This notebook simulates an attacker moving through a computer network from an entry point to a target database. The network is modelled as a weighted graph and seven AI search algorithms are implemented from scratch to find the attack path.

Algorithms covered: BFS, DFS, UCS, A*, Hill Climbing, Minimax, Alpha-Beta Pruning

✓ Step 1 — Install and Import Libraries

```
# run this cell first to make sure the libraries are available
!pip install networkx matplotlib --quiet
```

```
import heapq
import time
from collections import deque
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import networkx as nx

print('Libraries loaded successfully.')
```

Libraries loaded successfully.

✓ Step 2 — Network Definition

The network has 10 nodes (devices) and 16 edges (connections). Each edge has a weight that represents how easy it is to exploit that link. Lower weight = easier to attack.

Node	Device	Role
0	Entry Workstation	Start node
1	Router	Routing device
2	Switch	Layer-2 switch
3	Firewall	Security device
4	Web Server	Public server
5	Mail Server	Email gateway
6	Internal Network	Internal LAN
7	Database	Goal node (target)
8	Admin Workstation	Privileged machine
9	Backup Server	Backup storage

```
# node labels
NODES = {
    0: 'Entry WS',    1: 'Router',
    2: 'Switch',      3: 'Firewall',
    4: 'Web Server',  5: 'Mail Server',
    6: 'Internal Net', 7: 'Database',
    8: 'Admin WS',    9: 'Backup Srv'
}

# edges: (from_node, to_node, vulnerability_weight)
EDGES = [
    (0,1,2), (0,2,4), (1,2,1), (1,3,5), (1,4,7),
    (2,4,3), (2,5,6), (3,6,4), (3,8,5), (4,6,5),
    (4,9,4), (5,6,3), (6,7,8), (6,8,2), (8,7,3), (9,7,6)
]

# heuristic values for A* – estimated remaining cost to the database
HEURISTIC = {0:18, 1:14, 2:13, 3:10, 4:9, 5:9, 6:5, 7:0, 8:3, 9:7}
```

```

START = 0 # attacker entry point
GOAL = 7 # target database

def build_graph():
    graph = {i: [] for i in range(10)}
    for u, v, w in EDGES:
        graph[u].append((v, w))
        graph[v].append((u, w)) # undirected
    return graph

GRAPH = build_graph()

print(f'Graph ready: {len(NODES)} nodes, {len(EDGES)} edges')
print(f'Start: Node {START} ({NODES[START]})')
print(f'Goal: Node {GOAL} ({NODES[GOAL]})')

```

```

Graph ready: 10 nodes, 16 edges
Start: Node 0 (Entry WS)
Goal: Node 7 (Database)

```

Step 3 — Network Visualization

```

# fixed x,y positions for each node in the diagram
POS = {
    0:(0.0,0.5), 1:(1.5,0.78), 2:(1.5,0.22),
    3:(3.0,1.05), 4:(3.0,0.5), 5:(3.0,-0.05),
    6:(4.5,0.5), 7:(6.2,0.5), 8:(4.5,0.92), 9:(4.5,0.08)
}

def draw_network(path=None, title='Network Topology'):
    G = nx.Graph()
    for n in range(10):
        G.add_node(n)
    for u, v, w in EDGES:
        G.add_edge(u, v, weight=w)

    # find which edges are on the highlighted path
    path_edges = set()
    if path and len(path) > 1:
        for i in range(len(path) - 1):
            a = min(path[i], path[i+1])
            b = max(path[i], path[i+1])
            path_edges.add((a, b))

    # colour each node based on its role in the path
    node_colors = []
    for n in range(10):
        if path and n in path:
            if n == START:
                node_colors.append('#C0392B') # red for start
            elif n == GOAL:
                node_colors.append('#27AE60') # green for goal
            else:
                node_colors.append('#E67E22') # orange for path nodes
        else:
            node_colors.append('#AEB6BF') # grey for unvisited

    # thicker red edges for the path, thin grey for the rest
    edge_colors = []
    edge_widths = []
    for u, v in G.edges():
        key = (min(u,v), max(u,v))
        if key in path_edges:
            edge_colors.append('#C0392B')
            edge_widths.append(3.8)
        else:
            edge_colors.append('#D5D8DC')
            edge_widths.append(1.2)

    fig, ax = plt.subplots(figsize=(13, 5.5))
    ax.set_facecolor('#FAFAFA')

    nx.draw_networkx_edges(G, POS, ax=ax,
                           edge_color=edge_colors, width=edge_widths)
    nx.draw_networkx_nodes(G, POS, ax=ax,

```

```

        node_color=node_colors, node_size=1500,
        linewidths=1.5, edgecolors='#566573')
nx.draw_networkx_labels(G, POS, labels=NODES, ax=ax,
                        font_size=7, font_weight='bold', font_color='white')
nx.draw_networkx_edge_labels(
    G, POS, edge_labels={(u,v):w for u,v,w in EDGES},
    ax=ax, font_size=8.5,
    bbox=dict(boxstyle='round,pad=0.2', fc='white', alpha=0.85)
)

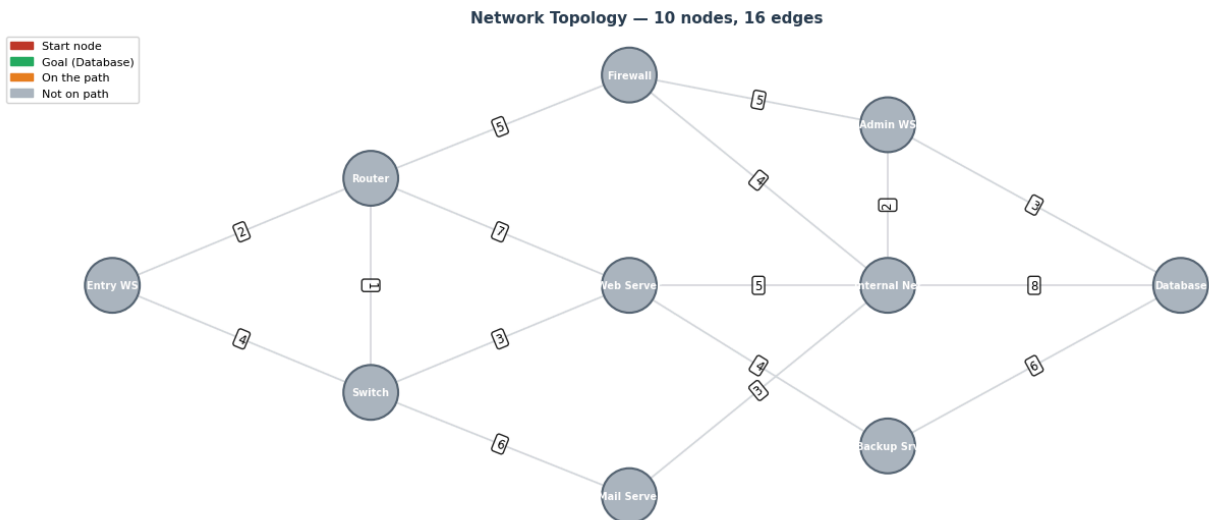
legend_handles = [
    mpatches.Patch(color='#C0392B', label='Start node'),
    mpatches.Patch(color='#27AE60', label='Goal (Database)'),
    mpatches.Patch(color='#E67E22', label='On the path'),
    mpatches.Patch(color='#AEB6BF', label='Not on path'),
]
ax.legend(handles=legend_handles, loc='upper left', fontsize=8)

if path:
    path_str = ' -> '.join(str(n) for n in path)
    ax.set_title(f'{title}\nPath: {path_str}',
                fontsize=11, fontweight='bold', color='#2C3E50', pad=10)
else:
    ax.set_title(title, fontsize=11, fontweight='bold', color='#2C3E50')

ax.axis('off')
plt.tight_layout()
plt.show()

# show the base network before running any algorithm
draw_network(title='Network Topology - 10 nodes, 16 edges')

```



Step 4 — Breadth-First Search (BFS)

BFS explores the network layer by layer using a queue (FIFO). It visits all nodes one hop away before going deeper. It finds the path with the fewest steps but does not consider edge weights, so the cost may not be minimum.

Complete: Yes **Optimal (cost):** No

```

def bfs(graph, start, goal):
    start_time = time.perf_counter()
    queue = deque([(start, [start], 0)]) # (current node, path so far, total cost)
    visited = set()
    expanded = 0

```

```

while queue:
    node, path, cost = queue.popleft()
    if node in visited:
        continue
    visited.add(node)
    expanded += 1

    if node == goal:
        ms = round((time.perf_counter() - start_time) * 1000, 4)
        return path, cost, expanded, ms

    for neighbor, weight in graph[node]:
        if neighbor not in visited:
            queue.append((neighbor, path + [neighbor], cost + weight))

return [], -1, expanded, 0

result_bfs = bfs(GRAPH, START, GOAL)
path, cost, exp, ms = result_bfs

print('BFS Results')
print('-' * 45)
print(f'Path: {path}')
print(f'Path (labels): {" -> ".join(NODES[n] for n in path)}')
print(f'Total cost: {cost}')
print(f'Nodes expanded: {exp}')
print(f'Time (ms): {ms}')

draw_network(path, title='BFS - Breadth-First Search')

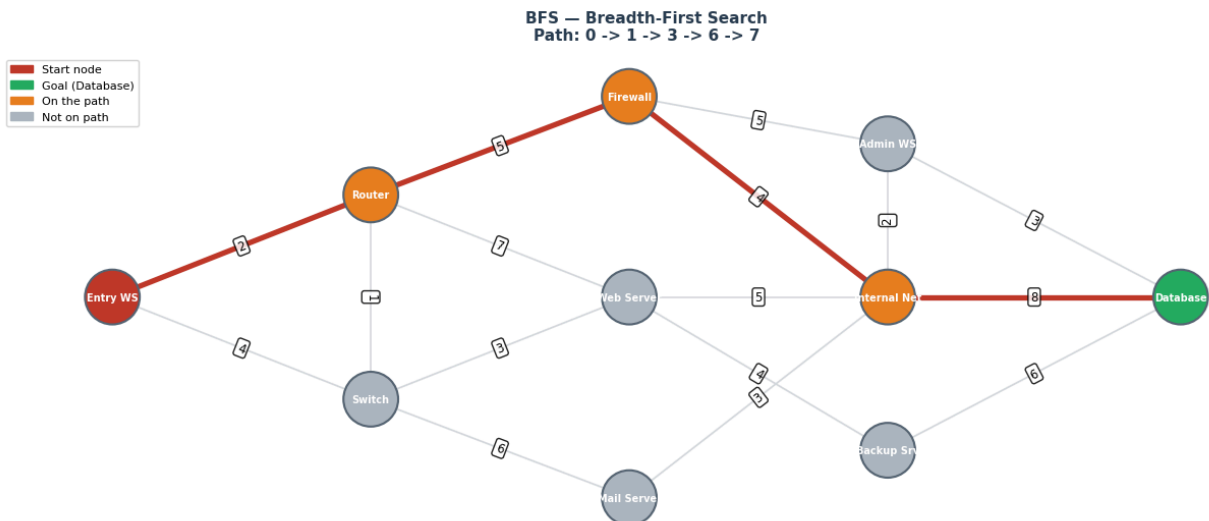
```

BFS Results

```

-----
Path: [0, 1, 3, 6, 7]
Path (labels): Entry WS -> Router -> Firewall -> Internal Net -> Database
Total cost: 19
Nodes expanded: 10
Time (ms): 0.0303

```



Step 5 — Depth-First Search (DFS)

DFS goes as deep as possible along one branch before backtracking. It uses a stack (LIFO). It usually finds a path quickly but it tends to be longer and more expensive than necessary.

Complete: Yes **Optimal:** No

```
def dfs(graph, start, goal):
    start_time = time.perf_counter()
    stack = [(start, [start], 0)]
    visited = set()
    expanded = 0

    while stack:
        node, path, cost = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        expanded += 1

        if node == goal:
            ms = round((time.perf_counter() - start_time) * 1000, 4)
            return path, cost, expanded, ms

        # reversed so we explore neighbors left to right
        for neighbor, weight in reversed(graph[node]):
            if neighbor not in visited:
                stack.append((neighbor, path + [neighbor], cost + weight))

    return [], -1, expanded, 0

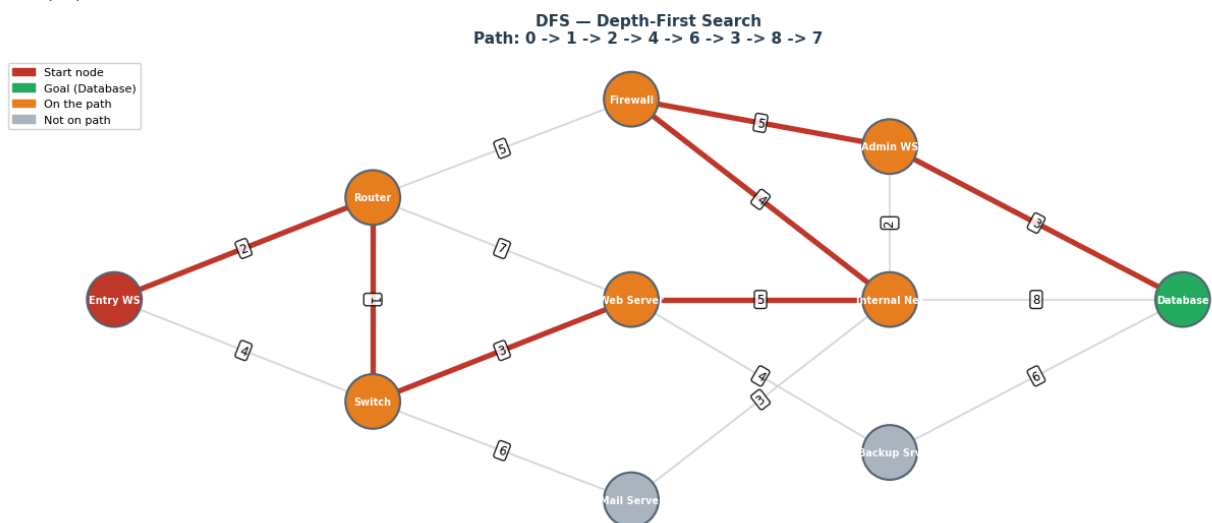
result_dfs = dfs(GRAPH, START, GOAL)
path, cost, exp, ms = result_dfs

print('DFS Results')
print('-' * 45)
print(f'Path: {path}')
print(f'Path (labels): {" -> ".join(NODES[n] for n in path)}')
print(f'Total cost: {cost}')
print(f'Nodes expanded: {exp}')
print(f'Time (ms): {ms}')

draw_network(path, title='DFS - Depth-First Search')
```

DFS Results

```
Path: [0, 1, 2, 4, 6, 3, 8, 7]
Path (labels): Entry WS -> Router -> Switch -> Web Server -> Internal Net -> Firewall -> Admin WS -> Database
Total cost: 23
Nodes expanded: 8
Time (ms): 0.0678
```



Step 6 — Uniform Cost Search (UCS)

UCS always expands the node with the lowest total cost from the start. It uses a min-heap priority queue. When it reaches the goal, the path is guaranteed to be the cheapest one possible.

Complete: Yes **Optimal:** Yes — always finds minimum cost path

```
def ucs(graph, start, goal):
    start_time = time.perf_counter()
    heap = [(0, start, [start])] # (cost, node, path)
    visited = set()
    expanded = 0

    while heap:
        cost, node, path = heapq.heappop(heap) # lowest cost first
        if node in visited:
            continue
        visited.add(node)
        expanded += 1

        if node == goal:
            ms = round((time.perf_counter() - start_time) * 1000, 4)
            return path, cost, expanded, ms

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                heapq.heappush(heap, (cost + weight, neighbor, path + [neighbor]))

    return [], -1, expanded, 0

result_ucs = ucs(GRAPH, START, GOAL)
path, cost, exp, ms = result_ucs

print('UCS Results')
print('-' * 45)
print(f'Path:           {path}')
print(f'Path (labels):   {" -> ".join(NODES[n] for n in path)}')
print(f'Total cost:      {cost} <-- optimal (lowest possible)')
print(f'Nodes expanded:  {exp}')
print(f'Time (ms):        {ms}')

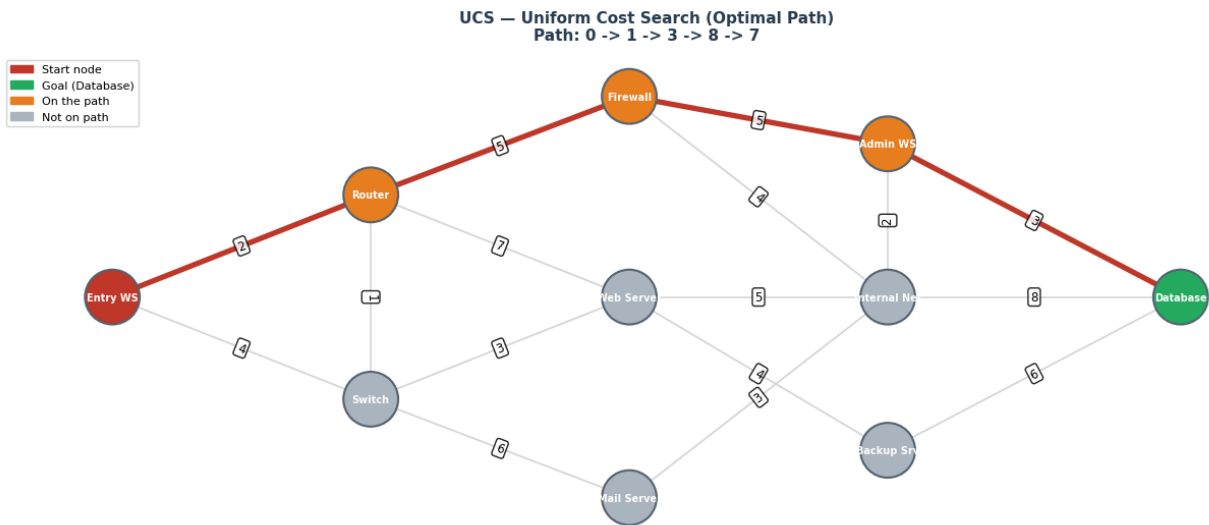
draw_network(path, title='UCS - Uniform Cost Search (Optimal Path)')
```

UCS Results

```

Path: [0, 1, 3, 8, 7]
Path (labels): Entry WS -> Router -> Firewall -> Admin WS -> Database
Total cost: 15 <-- optimal (lowest possible)
Nodes expanded: 10
Time (ms): 0.0382

```



Step 7 — A* Search

A* uses $f(n) = g(n) + h(n)$ where $g(n)$ is the actual cost from start to node n , and $h(n)$ is a heuristic estimate of the remaining cost to the goal. This lets it skip nodes that are unlikely to be on the best path.

The heuristic $h(n)$ was designed based on how many security layers each node still needs to pass through to reach the database and the minimum edge costs along those paths. It is admissible (never overestimates the true cost).

Complete: Yes **Optimal:** Yes (with admissible heuristic)

```

def astar(graph, start, goal, h):
    start_time = time.perf_counter()
    heap = [(h[start], 0, start, [start])] # (f, g, node, path)
    visited = set()
    expanded = 0

    while heap:
        f, g, node, path = heapq.heappop(heap)
        if node in visited:
            continue
        visited.add(node)
        expanded += 1

        if node == goal:
            ms = round((time.perf_counter() - start_time) * 1000, 4)
            return path, g, expanded, ms

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                new_g = g + weight
                new_f = new_g + h[neighbor] # f = g + h
                heapq.heappush(heap, (new_f, new_g, neighbor, path + [neighbor]))

    return [], -1, expanded, 0

```

```

print('Heuristic values h(n):')
for node, val in HEURISTIC.items():

```

```

print(f' Node {node} ({NODES[node]:15s}): h = {val}')

result_astar = astar(GRAPH, START, GOAL, HEURISTIC)
path, cost, exp, ms = result_astar

print()
print('A* Results')
print('-' * 45)
print(f'Path: {path}')
print(f'Path (labels): {" -> ".join(NODES[n] for n in path)}')
print(f'Total cost: {cost}')
print(f'Nodes expanded: {exp} <-- fewest of all algorithms')
print(f'Time (ms): {ms}')

draw_network(path, title='A* Search')

```

Heuristic values $h(n)$:

```

Node 0 (Entry WS) ): h = 18
Node 1 (Router) ): h = 14
Node 2 (Switch) ): h = 13
Node 3 (Firewall) ): h = 10
Node 4 (Web Server) ): h = 9
Node 5 (Mail Server) ): h = 9
Node 6 (Internal Net) ): h = 5
Node 7 (Database) ): h = 0
Node 8 (Admin WS) ): h = 3
Node 9 (Backup Srv) ): h = 7

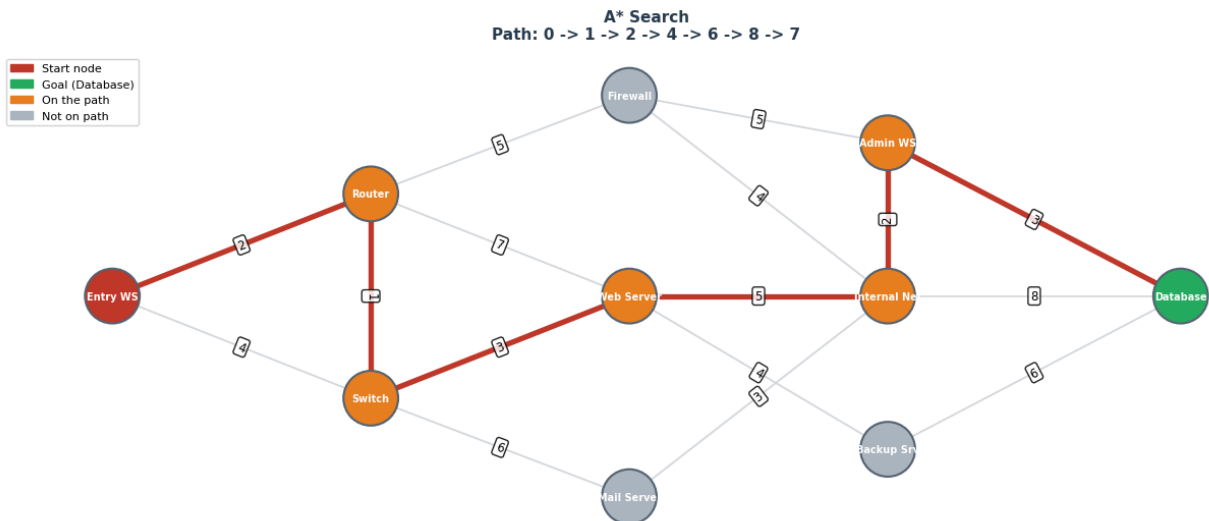
```

A* Results

```

-----
Path: [0, 1, 2, 4, 6, 8, 7]
Path (labels): Entry WS -> Router -> Switch -> Web Server -> Internal Net -> Admin WS -> Database
Total cost: 16
Nodes expanded: 7 <-- fewest of all algorithms
Time (ms): 0.0338

```



✓ Step 8 — Hill Climbing

Hill Climbing is a local search method. At each step it picks the neighbor with the lowest $h(n)$ and moves there. It cannot backtrack.

Local Maximum Problem: If every neighbor has a heuristic value that is equal to or worse than the current node, the algorithm gets stuck and stops — even if the goal has not been reached. This is demonstrated below.

Complete: No **Optimal:** No

```

def hill_climbing(graph, start, goal, h):
    start_time = time.perf_counter()
    current = start

```

```

path = [current]
cost = 0
expanded = 0
visited = {current}
status = 'goal reached'

while current != goal:
    expanded += 1
    candidates = [(nb, w) for nb, w in graph[current] if nb not in visited]

    if not candidates:
        status = f'stuck at node {current} - no more moves'
        break

    # greedy choice: move to neighbor with best (lowest) heuristic
    best_nb, best_w = min(candidates, key=lambda x: h[x[0]])

    # local maximum: best available neighbor is not better than current
    if h[best_nb] >= h[current] and current != start:
        status = f'stuck at local maximum - node {current} ({NODES[current]})'
        break

    visited.add(best_nb)
    path.append(best_nb)
    cost += best_w
    current = best_nb

ms = round((time.perf_counter() - start_time) * 1000, 4)
return path, cost, expanded, ms, status

result_hc = hill_climbing(GRAPH, START, GOAL, HEURISTIC)
path, cost, exp, ms, status = result_hc

print('Hill Climbing Results')
print('-' * 45)
print(f'Path: {path}')
print(f'Path (labels): {" -> ".join(NODES[n] for n in path)}')
print(f'Total cost: {cost}')
print(f'Nodes expanded: {exp}')
print(f'Time (ms): {ms}')
print(f'Status: {status}')

draw_network(path, title=f'Hill Climbing - Status: {status}')

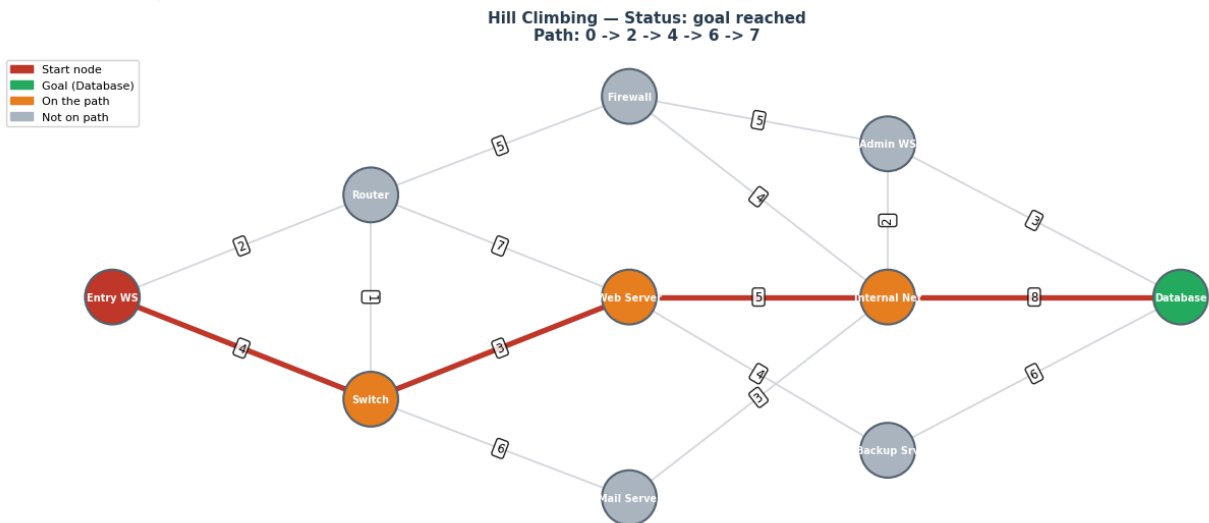
```

Hill Climbing Results

```

Path: [0, 2, 4, 6, 7]
Path (labels): Entry WS -> Switch -> Web Server -> Internal Net -> Database
Total cost: 20
Nodes expanded: 4
Time (ms): 0.0272
Status: goal reached

```



Step 9 — Minimax and Alpha-Beta Pruning

Minimax models an attacker vs defender scenario. The attacker (Maximiser) tries to reach the database, while the defender (Minimiser) tries to block the path. Both players are assumed to play optimally. The game tree is explored to depth 5.

Alpha-Beta Pruning is an optimisation of Minimax. It skips branches that cannot change the final outcome. Alpha (α) is the best score the attacker has found so far, and beta (β) is the best score the defender has found. If $\beta \leq \alpha$, the branch is pruned. The result is the same but fewer nodes are evaluated.

```

node_count = 0 # global counter to track how many nodes are evaluated

def minimax(graph, node, depth, is_max, goal, h, path, visited, a, b, pruning):
    global node_count
    node_count += 1

    if node == goal or depth == 0:
        return h[node], path

    neighbors = [(nb, w) for nb, w in graph[node] if nb not in visited]
    if not neighbors:
        return h[node], path

    best_path = path

    if is_max: # attacker's turn - maximize score
        best = float('-inf')
        for nb, _ in neighbors:
            val, p = minimax(graph, nb, depth-1, False, goal, h,
                             path+[nb], visited|{nb}, a, b, pruning)
            if val > best:
                best, best_path = val, p
        if pruning:
            a = max(a, best)
            if b <= a:
                break # beta cutoff - defender would not allow this
    return best, best_path

```

```

else:
    # defender's turn - minimize score
    best = float('inf')
    for nb, _ in neighbors:
        val, p = minimax(graph, nb, depth-1, True, goal, h,
                        path+[nb], visited|{nb}, a, b, pruning)
        if val < best:
            best, best_path = val, p
    if pruning:
        b = min(b, best)
        if b <= a:
            break # alpha cutoff - attacker would not allow this
    return best, best_path

def run_minimax(pruning=False):
    global node_count
    node_count = 0
    start_time = time.perf_counter()
    _, path = minimax(GRAPH, START, 5, True, GOAL, HEURISTIC,
                    [START], {START}, float('-inf'), float('inf'), pruning)
    cost = sum(
        next(w for nb, w in GRAPH[path[i]] if nb == path[i+1])
        for i in range(len(path)-1)
    )
    ms = round((time.perf_counter() - start_time) * 1000, 4)
    return path, cost, node_count, ms

# run Minimax without pruning
result_mm = run_minimax(pruning=False)
path_mm, cost_mm, exp_mm, ms_mm = result_mm

print('Minimax Results')
print('-' * 45)
print(f'Path: {path_mm}')
print(f'Path (labels): {" -> ".join(NODES[n] for n in path_mm)}')
print(f'Total cost: {cost_mm}')
print(f'Nodes evaluated:{exp_mm}')
print(f'Time (ms): {ms_mm}')

draw_network(path_mm, title='Minimax')

# run Alpha-Beta Pruning
result_ab = run_minimax(pruning=True)
path_ab, cost_ab, exp_ab, ms_ab = result_ab

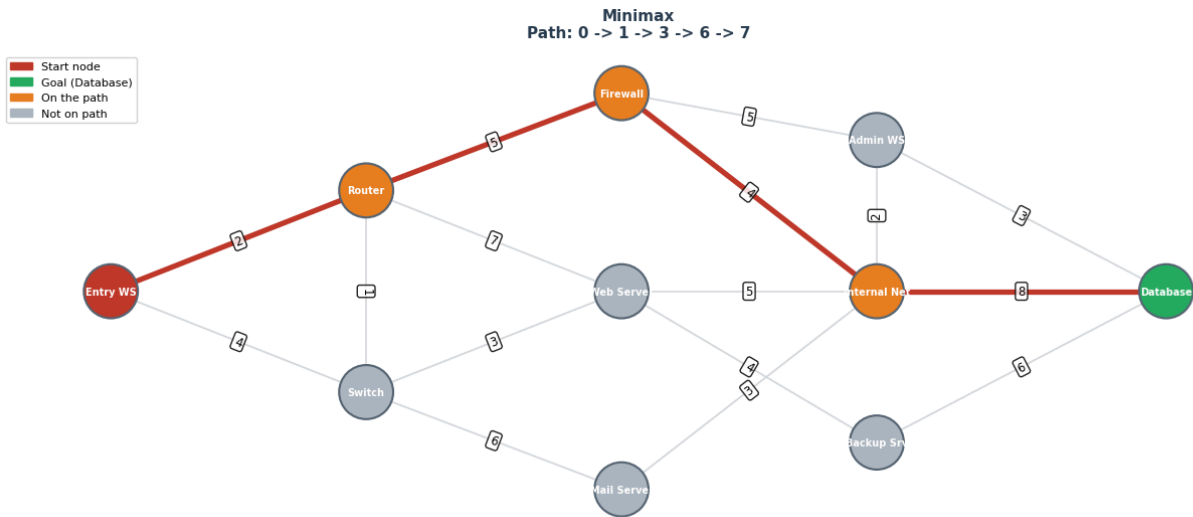
print()
print('Alpha-Beta Pruning Results')
print('-' * 45)
print(f'Path: {path_ab}')
print(f'Path (labels): {" -> ".join(NODES[n] for n in path_ab)}')
print(f'Total cost: {cost_ab}')
print(f'Nodes evaluated:{exp_ab}')
print(f'Time (ms): {ms_ab}')
reduction = round((1 - exp_ab / exp_mm) * 100, 1)
print(f'Pruning saved: {reduction}% fewer node evaluations than Minimax')

draw_network(path_ab, title='Alpha-Beta Pruning')

```

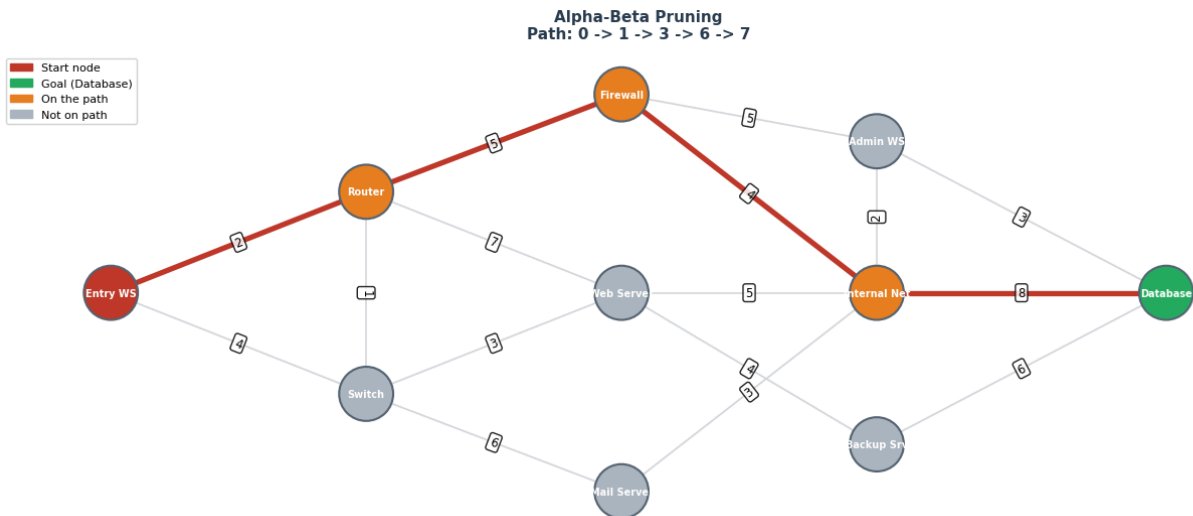
Minimax Results

Path: [0, 1, 3, 6, 7]
 Path (labels): Entry WS -> Router -> Firewall -> Internal Net -> Database
 Total cost: 19
 Nodes evaluated: 95
 Time (ms): 0.1255



Alpha-Beta Pruning Results

Path: [0, 1, 3, 6, 7]
 Path (labels): Entry WS -> Router -> Firewall -> Internal Net -> Database
 Total cost: 19
 Nodes evaluated: 66
 Time (ms): 0.2156
 Pruning saved: 30.5% fewer node evaluations than Minimax



Step 10 — Comparative Analysis

```

# collect all results
all_results = {
    'BFS': result_bfs,
    'DFS': result_dfs,
    'UCS': result_ucs,
    'A*': result_astar,
    'Hill Climbing': result_hc[:4], # first 4 values (no status field)
    'Minimax': result_mm,
    'Alpha-Beta': result_ab,
}

# Hill Climbing has 5 return values, trim to 4 for uniform handling
all_results['Hill Climbing'] = result_hc[:4]

SEP = '=' * 72
print(SEP)
print(' COMPARATIVE ANALYSIS TABLE')
print(SEP)
print(f'{"Algorithm":<18}{"Path":^28}{"Cost":>5} {"Nodes":>7} {"Time(ms)":>9}')
print('-' * 72)

for name, (path, cost, exp, ms) in all_results.items():
    p_str = '->'.join(map(str, path)) if path else 'N/A'
    # mark if Hill Climbing got stuck
    note = ''
    if name == 'Hill Climbing' and result_hc[4] != 'goal reached':
        note = ' [stuck]'
    print(f'{name:<18}{p_str:^28}{cost:>5} {exp:>7} {ms:>9}{note}')

print(SEP)
print()
print('Note: UCS finds the minimum cost path (optimal).')
print('Note: A* expands the fewest nodes among informed search algorithms.')
print('Note: Alpha-Beta evaluates fewer nodes than Minimax with the same result.')

```

```

=====
COMPARATIVE ANALYSIS TABLE
=====
Algorithm          Path          Cost    Nodes    Time(ms)
-----
BFS                0->1->3->6->7      19      10      0.0303
DFS                0->1->2->4->6->3->8->7  23       8      0.0678
UCS                0->1->3->8->7      15      10      0.0382
A*                0->1->2->4->6->8->7      16       7      0.0338
Hill Climbing      0->2->4->6->7       20       4      0.0272
Minimax            0->1->3->6->7      19      95      0.1255
Alpha-Beta         0->1->3->6->7      19      66      0.2156
=====

```

Note: UCS finds the minimum cost path (optimal).
Note: A* expands the fewest nodes among informed search algorithms.